



SRI SHANMUGHA
COLLEGE OF ENGINEERING AND TECHNOLOGY
AN AUTONOMOUS INSTITUTION , AFFILIATED TO ANNA UNIVERSITY
APPROVED BY AICTE,ACCREDITED BY NAAC WITH 'A' GRADE

DEPARTMENT OF BIOMEDICAL ENGINEERING



LABORATORY MANUAL

CS3691 – Embedded Systems and IoT Practical Exercises

(For III-year V Semester B.Tech – IT Students)

Regulation – 2021

NAME :

REGISTER NUMBER :

YEAR/SEMESTER :

BRANCH :

Prepared By: G.Vijaykumar M.E (Ph.D), AP/BME

CERTIFICATE

REGISTER NO.

**Certified that this is the bonafide record of work done by Mr/Ms
of the Semester in branch during the year
..... in the Laboratory.**

Staff In-charge

Head of the Department

Submitted for the Practical Examination held on _____

Internal Examiner

External Examiner

Name :
Year/Semester :

Reg.No.:

CONTENTS

EXP. NO.	DATE	NAME OF THE EXPERIMENT	PAGE NO.	MARKS	FACULTY SIGNATURE
1		Write 8051 Assembly Language experiments using simulator			
2		Test data transfer between registers and memory			
3		Perform ALU operations.			
4		Write Basic and arithmetic Programs Using Embedded C.			
5		Introduction to Arduino platform and programming			
6		Explore different communication methods with IoT devices (Zigbee, GSM, Bluetooth)			
7		Introduction to Raspberry PI platform and python programming			
8		Interfacing sensors with Raspberry PI			
9		Communicate between Arduino and Raspberry PI using any wireless medium			
10		Setup a cloud platform to log the data			
11		Log Data using Raspberry PI and upload to the cloud platform			
12		Design an IOT based system			

EXP. NO: 01

DATE:

WRITE 8051 ASSEMBLY LANGUAGE EXPERIMENTS USING SIMULATOR

Aim:

To write 8051 assembly language experiments using simulator.

SOFTWARE REQUIRED:

1. Keil uVision IDE

Theory:

Earlier to Microcontrollers, Microprocessors were greatly used for each and every purpose. Microprocessors were containing ALU, general purpose register, stack pointer, program counter, clock counter and so many other features which the today's Microcontroller also possesses. But the difference between them exists with respect to the number of instructions, access times, size, reliability, PCB size and so on. Microprocessor contains large instruction set called as CISC processor whereas Microcontroller contains less number of instructions and is called as RISC processor. The access time is less in case of microcontrollers compared to microprocessors and the PCB size reduces in case of microcontrollers.

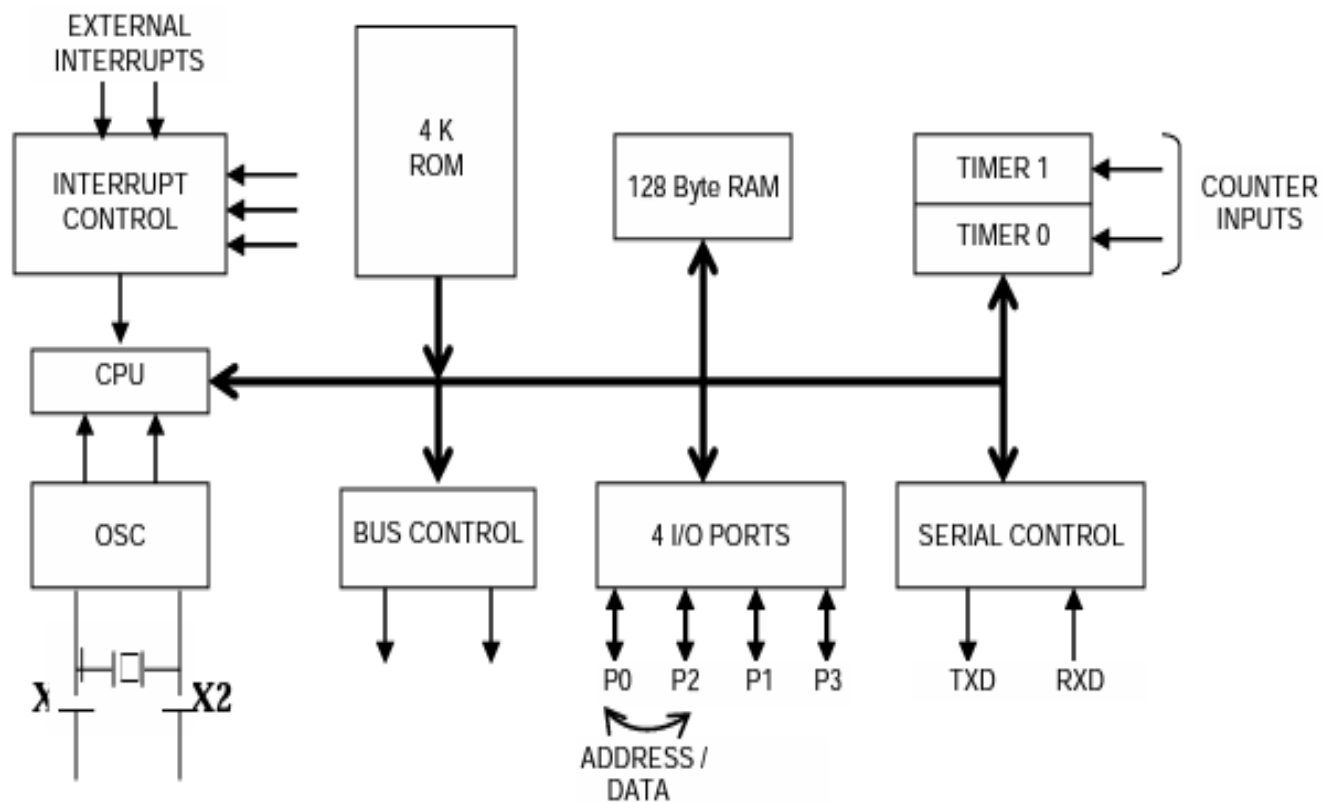
There are many versions of microcontrollers 8051, 8052, 8751, AT8951 from Atmel Corporation and many more. In this manual we will study about the 8051 architecture, its features, programming and interfacing.

MCS 8051 is an 8-bit single chip microcontroller with many built-in functions and is the core for all MCS-51 devices.

The main features of the 8051 core are:

- ♣ Operates with single Power Supply +5V.
- ♣ 8-bit CPU optimized for control applications.
- ♣ 16-bit program counter (PC) and 16-bit data pointer (DPTR).
- ♣ 8-bit program status word (PSW).
- ♣ 8-bit stack pointer (SP).
- ♣ 4K Bytes of On-Chip Program Memory (Internal ROM or EPROM).
- ♣ 128 bytes of On-Chip Data Memory (Internal RAM):

- ◆ Four Register Banks, each containing 8 registers (R0 to R7)
[Total 32 reg]
- ◆ 16-bytes of bit addressable memory.
- ◆ 80 bytes of general-purpose data memory (Scratch Pad Area).
- ♣ Special Function Registers (SFR) to configure/operate microcontroller.
- ♣ 32 bit bi-directional I/O Lines (4 ports P0 to P3).
- ♣ Two 16-bit timers/counters (T0 and T1).
- ♣ Full duplex UART (Universal Asynchronous Receiver/Transmitter).
- ♣ On-Chip oscillator and clock circuitry.

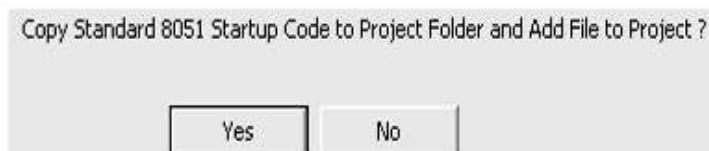


General Block Diagram of 8051 Microcontroller Architecture

STEPS TO CREATE AND COMPILE Keil μ Vision-3/4 PROJECT:



1. Double Click on the μ Vision3/4 icon on the desktop.
2. Close any previous projects that were opened using – **Project -> Close**.
3. Start **Project – New Project**, and select the CPU from the device database (Database-Atmel- AT89C51ED2 or AT89C51RD2 as per the board).On clicking ‘OK’, the following option is displayed. Choose ‘No’.

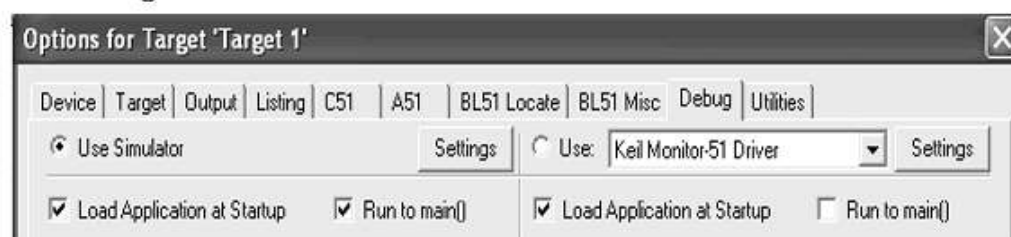


4. Create a source file (using **File->New**), type in the assembly or C program and save this (filename.asm/filename.c) and add this source file to the project using either one of the following two methods. (i) **Project->Manage->Components, Environment Books->addfiles->** browse to the required file -> **OK**

“OR” ii) right click on the Source Group in the Project Window and the **Add Files to Group** option.



5. Set the Target options using -> **Project – Options for Target** opens the μ Vision2 **Options for Target – Target** configuration dialog. Set the **Xtal** (Crystal frequency) frequency as 11.0592 MHz, and also the **Options for Target – Debug – use either Simulator / Keil Monitor- 51 driver**.



6. If **Keil Monitor- 51 driver is used click on Settings** -> COM Port settings select the COM Port to which the board is connected and select the baud rate as 19200 or 9600 (recommended). Enable **Serial Interrupt** option if the user application is not using on-chip UART, to stop program execution.
7. Build the project; using **Project -> Build Project**. μ Vision translates all the user application and links. Any errors in the code are indicated by – “Target not created” in the Build window, along with the error line. Debug the errors. After an error free, to build go to Debug mode.

```

x Build target 'Target 1'
  assembling STARTUP.A51...
  assembling BLKMOV.ASM...
  BLKMOV.ASM(10): error A45: UNDEFINED SYMBOL (PASS-2)
  Target not created
  
```

8. Now user can enter into **Debug** mode with **Debug- Start / Stop Debug session** dialog. Or by clicking in the  icon.
9. The program is run using the **Debug-Run** command & halted using **Debug-Stop** **Running**. Also the    (reset, run, halt) icons can be used. Additional icons are     (step, step over, and step into, run till cursor).
10. If it is an interface program the outputs can be seen on the LCD, CRO, motor, led status, etc. If it is a part-A program, the appropriate memory window is opened using View -> memory window (for data RAM & XRAM locations), Watch window (for timer program), serial window, etc.
11. **Note:** To access data RAM area type address as D: 0020h. Similarly to access the DPTR region (XRAM-present on chip in AT89C51ED2) say 9000h location type in X: 09000H.

Procedure:

1. Launch the Simulator
2. Write the Assembly Program and save it with .asm extension
3. Click the build target option from project menu to assemble the program
4. Click the Debug menu to Start/Stop Debug Session
5. Press STEP button to single step the Program

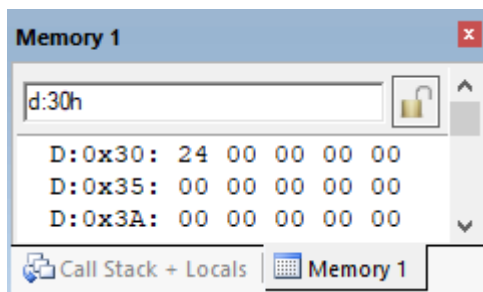
8051 Assembly Language Experiments

- a. Write an assembly language program to find the square of a given number N.

Let N = 06

```
mov a,#06    // a=N=06
mov b,a
mul ab
mov 30h,a    // result is stored in 30h and 31h
mov 31h,b
end
```

Result:



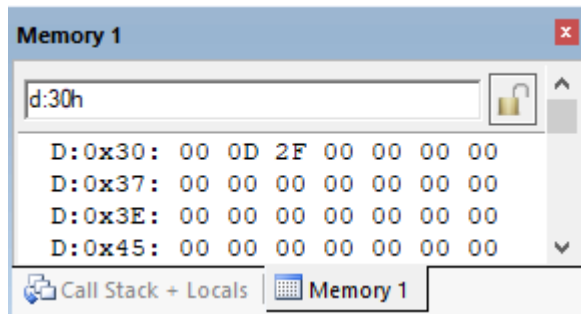
- b. Write an assembly language program to find the cube of a given number.

```
mov r0,#0fh  // r0 = given number to find the cube of it.
mov a,r0
mov b,r0
mul ab
mov r1,b
mov b,r0
mul ab
mov 32h,a
mov r2,b
mov a,r1
mov b,r0
mul ab
add a,r2
mov 31h,a
mov a,b
```



```
mov 30h,a      //result is stored in 30h, 31h, 32h end
```

Result:



c. Write an assembly language program to perform logical operations AND, OR, XOR on two eight bit numbers -45h & 67h

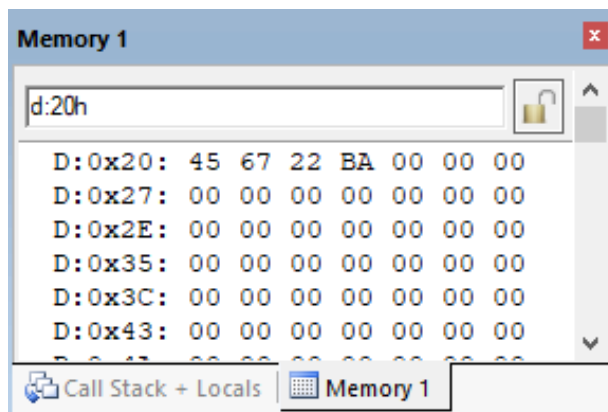
```
// Logical operations between two 8-bit numbers//
```

```

mov a,#45h      //1st number 0100 0101
mov r0,#67h     //2nd number 0110 0111
anl a,r0        //AND Logic  0100 0101
mov 20h,a       // saving and result with 20h memory
mov a,#45h
orl a,r0        //OR Logic   0110 0111
mov 21h,a       // saving and result with 21h memory
mov a,#45h
xrl a,r0        //XOR Logic  0010 0010
mov 22h,a       // saving and result with 22h memory
mov a,#45h
cpl a           //NOT LOGIC  1011 1010
mov 23h,a       // saving and result with 23h memory
end

```

Result:



Result:

Thus the 8051 assembly language experiments using Keil IDE was executed and simulation results were verified.

Test data transfer between registers and memory**Aim:**

To write 8051 assembly language program to test data transfer between register and memory

SOFTWARE REQUIRED:

1. Keil uVision IDE

Theory:

Earlier to Microcontrollers, Microprocessors were greatly used for each and every purpose. Microprocessors were containing ALU, general purpose register, stack pointer, program counter, clock counter and so many other features which the today's Microcontroller also possesses. But the difference between them exists with respect to the number of instructions, access times, size, reliability, PCB size and so on. Microprocessor contains large instruction set called as CISC processor whereas Microcontroller contains less number of instructions and is called as RISC processor. The access time is less in case of microcontrollers compared to microprocessors and the PCB size reduces in case of microcontrollers.

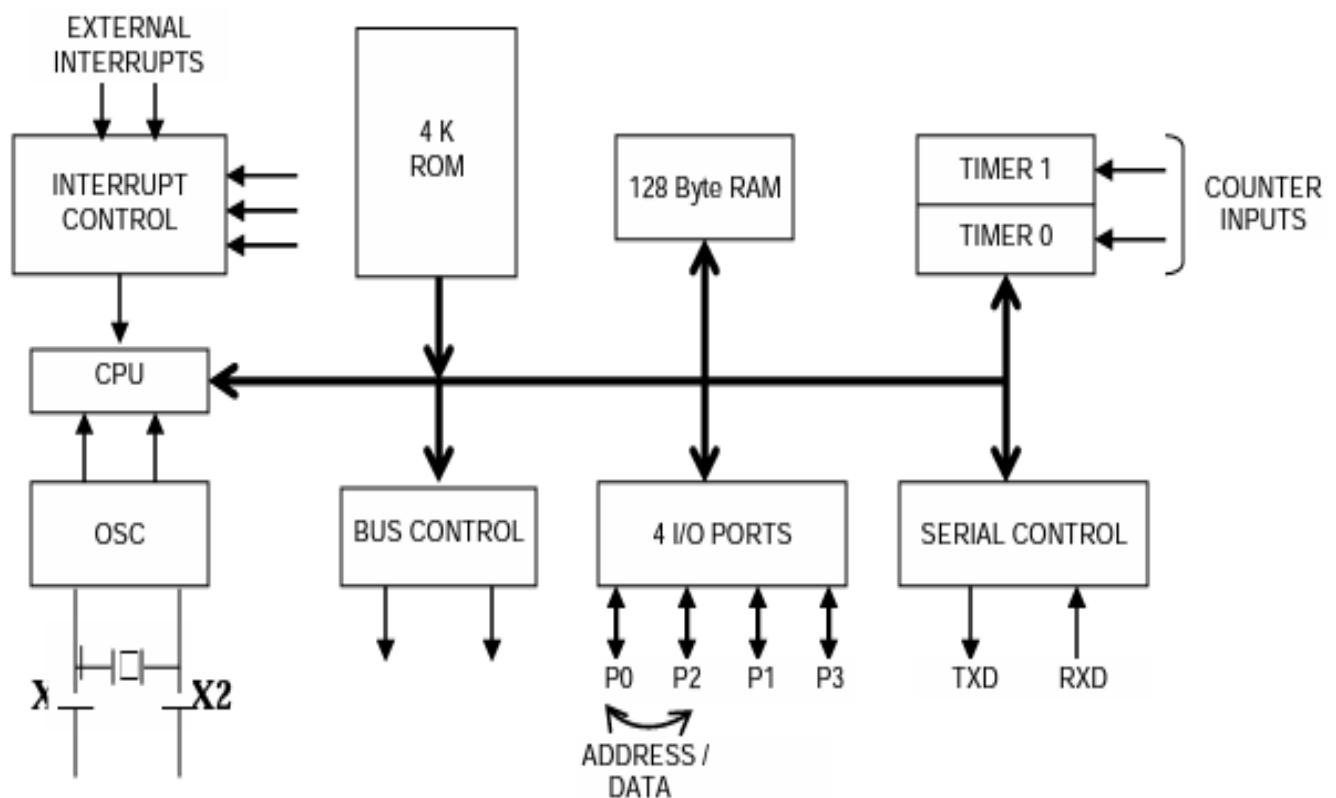
There are many versions of microcontrollers 8051, 8052, 8751, AT8951 from Atmel Corporation and many more. In this manual we will study about the 8051 architecture, its features, programming and interfacing.

MCS 8051 is an 8-bit single chip microcontroller with many built-in functions and is the core for all MCS-51 devices.

The main features of the 8051 core are:

- ♣ Operates with single Power Supply +5V.
- ♣ 8-bit CPU optimized for control applications.
- ♣ 16-bit program counter (PC) and 16-bit data pointer (DPTR).
- ♣ 8-bit program status word (PSW).
- ♣ 8-bit stack pointer (SP).
- ♣ 4K Bytes of On-Chip Program Memory (Internal ROM or EPROM).

- ♣ 128 bytes of On-Chip Data Memory (Internal RAM):
 - ◆ Four Register Banks, each containing 8 registers (R0 to R7)
[Total 32 reg]
 - ◆ 16-bytes of bit addressable memory.
 - ◆ 80 bytes of general-purpose data memory (Scratch Pad Area).
- ♣ Special Function Registers (SFR) to configure/operate microcontroller.
- ♣ 32 bit bi-directional I/O Lines (4 ports P0 to P3).
- ♣ Two 16-bit timers/counters (T0 and T1).
- ♣ Full duplex UART (Universal Asynchronous Receiver/Transmitter).
- ♣ On-Chip oscillator and clock circuitry.

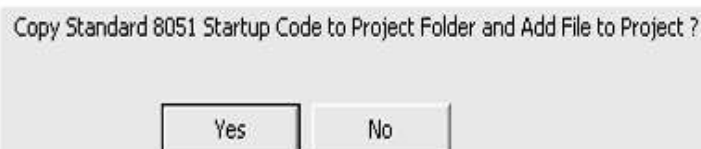


General Block Diagram of 8051 Microcontroller Architecture

STEPS TO CREATE AND COMPILE Keil μ Vision-3/4 PROJECT:



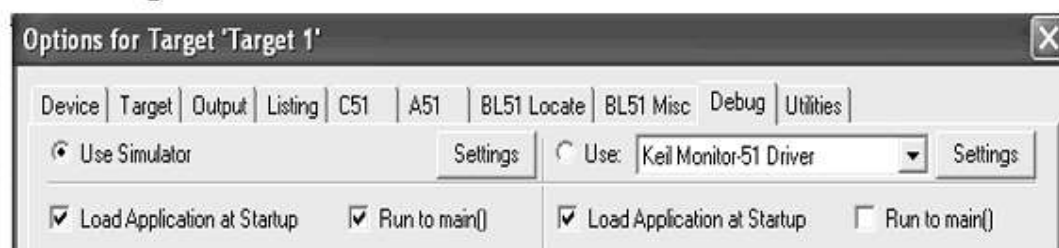
1. Double Click on the μ Vision3/4 icon on the desktop.
2. Close any previous projects that were opened using – **Project -> Close**.
3. Start **Project – New Project**, and select the CPU from the device database (Database-Atmel- AT89C51ED2 or AT89C51RD2 as per the board).On clicking ‘OK’, the following option is displayed. Choose ‘No’.



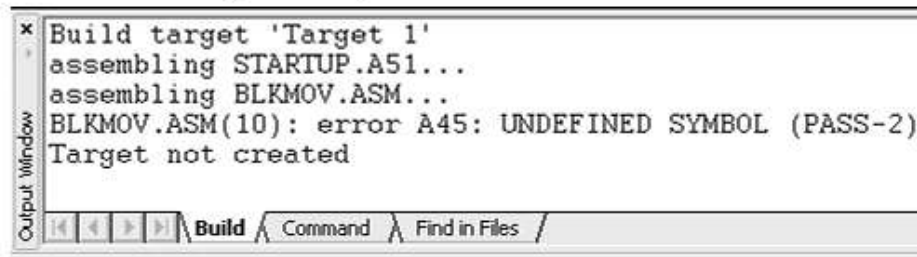
4. Create a source file (using **File->New**), type in the assembly or C program and save this (filename.asm/filename.c) and add this source file to the project using either one of the following two methods. (i) **Project->Manage->Components, Environment Books->addfiles->** browse to the required file -> **OK**
“OR” ii) right click on the Source Group in the Project Window and the **Add Files to Group** option.



5. Set the Target options using -> **Project – Options for Target** opens the μ Vision2 **Options for Target – Target** configuration dialog. Set the **Xtal** (Crystal frequency) frequency as 11.0592 MHz, and also the **Options for Target – Debug – use either Simulator / Keil Monitor- 51 driver**.



6. If **Keil Monitor- 51 driver is used click on Settings -> COM Port settings** select the COM Port to which the board is connected and select the baud rate as 19200 or 9600 (recommended). Enable **Serial Interrupt** option if the user application is not using on-chip UART, to stop program execution.
7. Build the project; using **Project -> Build Project**. μ Vision translates all the user application and links. Any errors in the code are indicated by – “Target not created” in the Build window, along with the error line. Debug the errors. After an error free, to build go to Debug mode.



8. Now user can enter into **Debug** mode with **Debug- Start / Stop Debug session** dialog. Or by clicking in the  icon.
9. The program is run using the **Debug-Run** command & halted using **Debug-Stop** **Running**. Also the    (reset, run, halt) icons can be used. Additional icons are     (step, step over, and step into, run till cursor).
10. If it is an interface program the outputs can be seen on the LCD, CRO, motor, led status, etc. If it is a part-A program, the appropriate memory window is opened using View -> memory window (for data RAM & XRAM locations), Watch window (for timer program), serial window, etc.
11. **Note:** To access data RAM area type address as D: 0020h. Similarly to access the DPTR region (XRAM-present on chip in AT89C51ED2) say 9000h location type in X: 09000H.

Procedure:

1. Launch the Simulator
2. Write the Assembly Program and save it with .asm extension
3. Click the build target option from project menu to assemble the program
4. Click the Debug menu to Start/Stop Debug Session
5. Press STEP button to single step the Program

Data Transfer Programming:

- a. Write an assembly language program to transfer N = ____ bytes of data from location A: ____h to location B: ____h.

Let $N = 05h$, $A: 30h$ $B: 40h$

```
mov r0,#30h           //source address
mov r1,#40h           //destination address
mov r7,#05h           //Number of bytes to be moved
```

```
back: mov a,@r0
      mov @r1,a
      inc r0
      inc r1
      djnz r7,back           //repeat till all data transferred
      end
```

Result:

Before Execution:

The screenshot shows two memory dump windows from the WinBox debugger. The top window is for address `d:30h` and the bottom window is for address `d:40h`. Both windows show a hex dump of memory with corresponding ASCII values. The bottom window highlights the first four memory slots (addresses `D:0x40` to `D:0x4D`) with a blue selection bar.

Address	Hex	ASCII
D:0x30	01 02 03 04 05 00 00 00 00 00	
D:0x3A	00 00 00 00 00 00 01 02 03 04	
D:0x44	05 00 00 00 00 00 00 00 00 00	
D:0x4E	00 00 00 00 00 00 00 00 00 00	
D:0x50	00 00 00 00 00 00 00 00 00 00	
D:0x40	00 00 00 00 00 00 00 00 00 00	
D:0x4A	00 00 00 00 00 00 00 00 00 00	
D:0x4C	00 00 00 00 00 00 00 00 00 00	
D:0x4E	00 00 00 00 00 00 00 00 00 00	
D:0x50	00 00 00 00 00 00 00 00 00 00	

After Execution:

[illegible]

- b. Write an assembly language program to find the largest element in a given array of N = ___ h bytes at location 4000h. Store the largest element at location 4062h.

Let N = 06h

```

mov r3,#6           //length of the array
mov dptr,#4000H     //starting address of array
movx a,@dptr
mov r1,a

nextbyte: inc dptr
movx a,@dptr
clr c               //reset borrow flag
mov r2,a            //next number in the array
subb a,r1           //other Num-Prev largest no.
jc skip            // JNC FOR SMALLEST ELEMENT
mov a,r2            //update larger number in r1
mov r1,a

skip: djnz r3,nextbyte
mov dptr, #4062H    //location of the result-4062h
mov a,r1            //largest number
movx @dptr,a        //store at #4062H
end

```

Result:

Before Execution:

Memory 1									
Address:		x:4000h							
X:0x004000:	01	02	03	09	FF	66	00		
X:0x004025:	00	00	00	00	00	00	00		

After Execution:

Memory 2									
Address:		x:4062h							
X:0x004062:	FF	00	00	00	00	00	00		
X:0x004087:	00	00	00	00	00	00	00		

C. Write an assembly language program to sort an array of N = ____ h bytes of data in ascending/descending order stored from location 9000h. (Using bubble sort algorithm)

Let N = 06h

```

        mov r0,#05H           //count (N-1)      array size = N
loop1:  mov dptr, #9000h       //array stored from address 9000h
        mov r1,#05h           //initialize exchange counter
loop2:  movx a, @dptr          //get number from array and store in B register
        mov b, a
        inc dptr
        movx a, @dptr          //next number in the array
        clr c                  //reset borrow flag
        mov r2, a              //store in R2
        subb a, b              //2nd-1st No, since no compare instruction in 8051
        jnc noexchg            //JC - FOR DESCENDING ORDER
        mov a,b                //exchange the 2 nos in the array
        movx @dptr,a
        dec dpl                //DEC DPTR - instruction not present
        mov a,r2
        movx @dptr,a
        inc dptr

noexchg: djnz r1,loop2          //decrement compare counter
        djnz r0,loop1          //decrement pass counter
        end

```

Result:

Before Execution:

Memory 1									
Address:		x:9000h							
X:0x009000:	66	55	44	33	22	11	00	00	
X:0x009025:	00	00	00	00	00	00	00	00	
X:0x00904A:	00	00	00	00	00	00	00	00	

After Execution :(Ascending order)

Memory 1									
Address:		x:9000h							
X:0x009000:	11	22	33	44	55	66	00	00	
X:0x009025:	00	00	00	00	00	00	00	00	
X:0x00904A:	00	00	00	00	00	00	00	00	

Result:

Thus the 8051 Assembly Language Programs to test the data transfer between memory and Registers were executed successfully.

PERFORM ALU OPERATIONS**Aim:**

To write 8051 assembly language program to perform ALU operations.

SOFTWARE REQUIRED:

1. Keil uVision IDE

Theory:

Earlier to Microcontrollers, Microprocessors were greatly used for each and every purpose. Microprocessors were containing ALU, general purpose register, stack pointer, program counter, clock counter and so many other features which the today's Microcontroller also possesses. But the difference between them exists with respect to the number of instructions, access times, size, reliability, PCB size and so on. Microprocessor contains large instruction set called as CISC processor whereas Microcontroller contains less number of instructions and is called as RISC processor. The access time is less in case of microcontrollers compared to microprocessors and the PCB size reduces in case of microcontrollers.

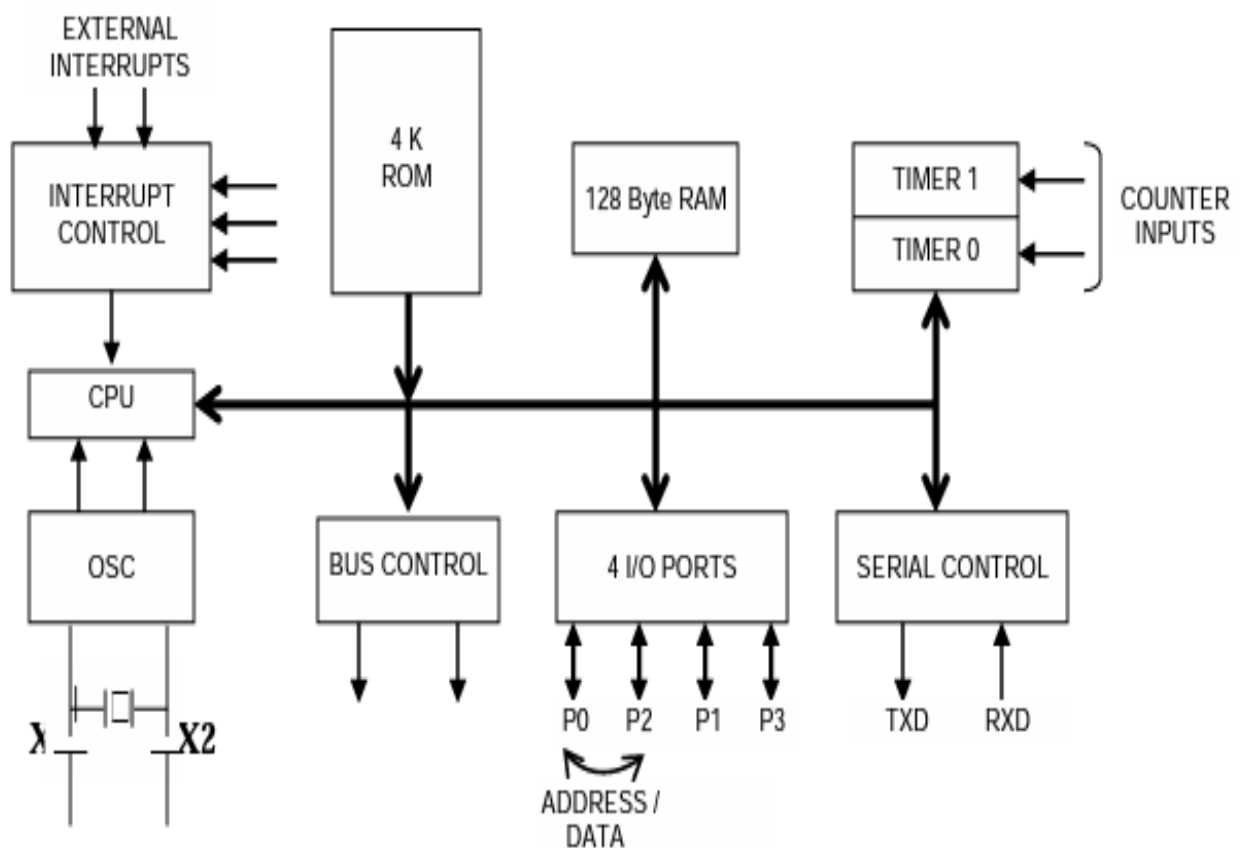
There are many versions of microcontrollers 8051, 8052, 8751, AT8951 from Atmel Corporation and many more. In this manual we will study about the 8051 architecture, its features, programming and interfacing.

MCS 8051 is an 8-bit single chip microcontroller with many built-in functions and is the core for all MCS-51 devices.

The main features of the 8051 core are:

- ♣ Operates with single Power Supply +5V.
- ♣ 8-bit CPU optimized for control applications.
- ♣ 16-bit program counter (PC) and 16-bit data pointer (DPTR).
- ♣ 8-bit program status word (PSW).
- ♣ 8-bit stack pointer (SP).
- ♣ 4K Bytes of On-Chip Program Memory (Internal ROM or EPROM).

- ♣ 128 bytes of On-Chip Data Memory (Internal RAM):
 - ◆ Four Register Banks, each containing 8 registers (R0 to R7)
[Total 32 reg]
 - ◆ 16-bytes of bit addressable memory.
 - ◆ 80 bytes of general-purpose data memory (Scratch Pad Area).
- ♣ Special Function Registers (SFR) to configure/operate microcontroller.
- ♣ 32 bit bi-directional I/O Lines (4 ports P0 to P3).
- ♣ Two 16-bit timers/counters (T0 and T1).
- ♣ Full duplex UART (Universal Asynchronous Receiver/Transmitter).
- ♣ On-Chip oscillator and clock circuitry.

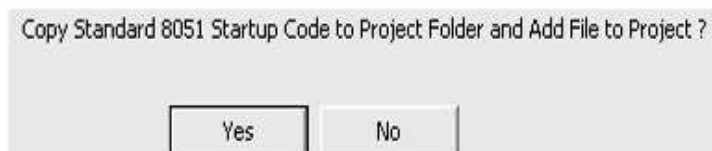


General Block Diagram of 8051 Microcontroller Architecture

STEPS TO CREATE AND COMPILE Keil μ Vision-3/4 PROJECT:



1. Double Click on the μ Vision3/4 icon on the desktop.
2. Close any previous projects that were opened using – **Project -> Close**.
3. Start **Project – New Project**, and select the CPU from the device database (Database-Atmel- AT89C51ED2 or AT89C51RD2 as per the board).On clicking ‘OK’, the following option is displayed. Choose ‘No’.

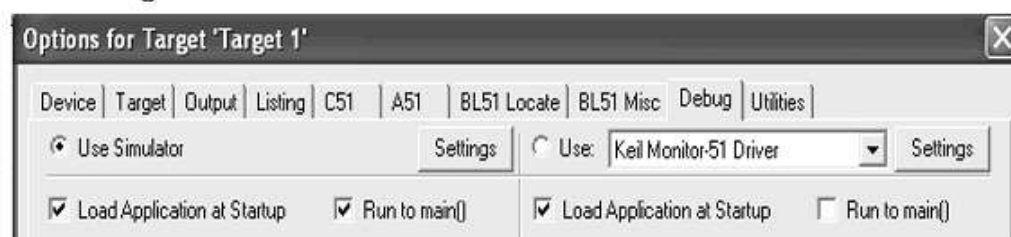


4. Create a source file (using **File->New**), type in the assembly or C program and save this (filename.asm/filename.c) and add this source file to the project using either one of the following two methods. (i) **Project->Manage->Components, Environment Books->addfiles->** browse to the required file -> **OK**

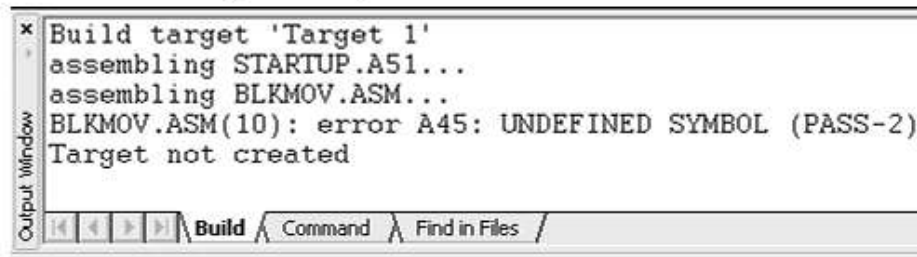
“OR” ii) right click on the Source Group in the Project Window and the **Add Files to Group** option.



5. Set the Target options using -> **Project – Options for Target** opens the μ Vision2 **Options for Target – Target** configuration dialog. Set the **Xtal** (Crystal frequency) frequency as 11.0592 MHz, and also the **Options for Target – Debug – use either Simulator / Keil Monitor- 51 driver**.



6. If **Keil Monitor- 51 driver is used click on Settings** -> COM Port settings select the COM Port to which the board is connected and select the baud rate as 19200 or 9600 (recommended). Enable **Serial Interrupt** option if the user application is not using on-chip UART, to stop program execution.
7. Build the project; using **Project -> Build Project**. μ Vision translates all the user application and links. Any errors in the code are indicated by – “Target not created” in the Build window, along with the error line. Debug the errors. After an error free, to build go to Debug mode.



```
Build target 'Target 1'
assembling STARTUP.A51...
assembling BLKMOV.ASM...
BLKMOV.ASM(10): error A45: UNDEFINED SYMBOL (PASS-2)
Target not created
```

8. Now user can enter into **Debug** mode with **Debug- Start / Stop Debug session** dialog. Or by clicking in the  icon.
9. The program is run using the **Debug-Run** command & halted using **Debug-Stop** **Running**. Also the    (reset, run, halt) icons can be used. Additional icons are     (step, step over, and step into, run till cursor).
10. If it is an interface program the outputs can be seen on the LCD, CRO, motor, led status, etc. If it is a part-A program, the appropriate memory window is opened using View -> memory window (for data RAM & XRAM locations), Watch window (for timer program), serial window, etc.
11. **Note:** To access data RAM area type address as D: 0020h. Similarly to access the DPTR region (XRAM-present on chip in AT89C51ED2) say 9000h location type in X: 09000H.

Procedure:

1. Launch the Simulator
2. Write the Assembly Program and save it with .asm extension
3. Click the build target option from project menu to assemble the program
4. Click the Debug menu to Start/Stop Debug Session
5. Press STEP button to single step the Program

Arithmetic Instruction Programming

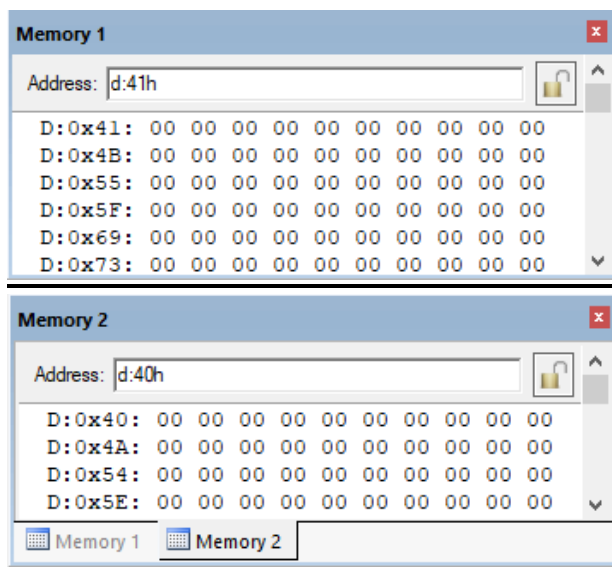
- a. Write an assembly language program to perform the addition of two 16-bit numbers.

/program to add 16bit numbers and storing the result in 40h and 41h location

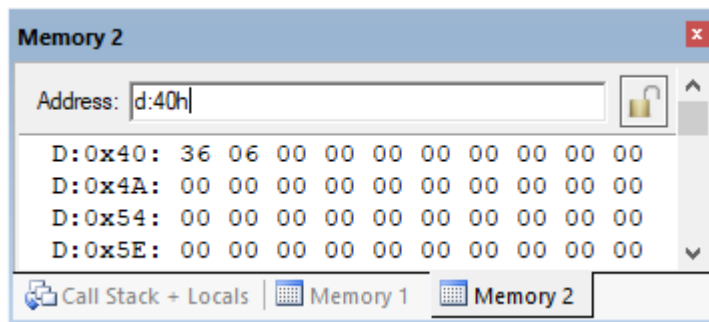
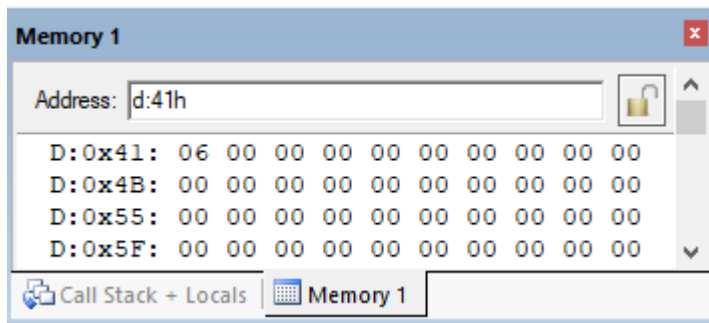
```
// 1173
/// +2493
/// -----
// 3606
org 00h
mov a, #73h
mov r0, #93h
add a, r0
mov 41h,a
mov a, #11h
mov r0, #24h
add a, r0
mov 40h,a
end
```

Result:

Before Execution:



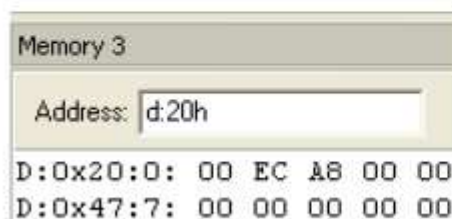
After Execution:



- b. Write an assembly language program to perform the subtraction of two 16-bit numbers.

```
mov r0,#0dch          //lower nibble of No.1
mov r1,#0feh          //higher nibble of No.1
mov r2,#34h           //lower nibble of No.2
mov r3,#12h           //higher nibble of No.2
clr c                 //
mov a,r0
subb a,r2
mov 22h,a
mov a,r1
subb a,r3
mov 21h,a
mov 00h,c
end
```

Result:



c. Write an assembly language program to perform the multiplication of two 16-bit numbers.

```
mov r0,#34h      // 5678*1234
mov r1,#12h
mov r2,#78h
mov r3,#56h
mov a,r0
mov b,r2
mul ab
mov 33h,a
mov r4,b
mov a,r0
mov b,r3
mul ab
add a,r4
mov r5,a
mov a,b
addc a,#00h
mov r6,a
mov a,r1
mov b,r2
mul ab
add a,r5
mov 32h,a
mov a,b
addc a,r6
mov 00h,c
mov r7,a
mov a,r3
mov b,r1
mul ab
add a,r7
mov 31h,a
mov a,b
addc a,20h
mov 30h,a
end
```


Result After Execution:

x		Address: d:30h									
Memory	D:0x30:	06	26	00	60	00	00	00	00	00	00
	D:0x41:	00	00	00	00	00	00	00	00	00	00
	D:0x52:	00	00	00	00	00	00	00	00	00	00
	D:0x63:	00	00	00	00	00	00	00	00	00	00
		Memory #1 Memory #2 Memory #3									

Result:

Thus the 8051 Assembly Language Program to perform ALU operation is executed successfully.

Ex no:04	Basic and arithmetic Programs Using Embedded C
Date:	

Aim

To write Basic and arithmetic Programs Using Embedded C

Apparatus Required

1. Personal computer
2. Arduino Uno board
3. Arduino simulator

Procedure

- Data Types: Embedded systems typically support various data types, such as integers (signed and unsigned), floating-point numbers, and fixed-point numbers. The choice of data type depends on factors like the required range, precision, and memory constraints.
- Binary Representation: Arithmetic operations in embedded systems are performed using binary representation. Numbers are typically stored in binary format using a fixed number of bits. For example, an 8-bit microcontroller can store values from 0 to 255.
- Integer Arithmetic: Embedded systems often deal with integer arithmetic. Addition and subtraction of integers are straightforward and can be performed using binary addition and subtraction algorithms. Multiplication and division operations may require more complex algorithms like shifting and bit manipulation.

Program

Arithmetic Program

```
void setup() {  
    // Initialize Serial  
  
    // Communication  
    Serial.begin(57600);  
}  
void loop() {  
    int num1 = 10; // First  
  
    // number  
    int num2 = 5; // Second  
  
    //number
```

```

    int result; // Result
    // Perform arithmetic
    //operations
    result = num1 + num2; // Addition
    Serial.print("Sum: ");
    Serial.println(result);
    result = num1 - num2;
    // Subtraction
    Serial.print("Difference: ");
    Serial.println(result);
    result = num1 * num2;
    //multiplication
    Serial.print("Product: ");
    Serial.println(result);
    result = num1 / num2;
    // Division
    Serial.print("Quotient: ");
    Serial.println(result);
    delay(1000);
    // Delay for 1 second
}

```

Output:

Program to Find the Sum of Two Numbers:

```

void setup() {
    // Initialize serial communication for debugging (optional)
    Serial.begin(57600);
}

void loop() {
    // Define two variables to hold the numbers
    int num1 = 10;
    int num2 = 20;

    // Find the sum of the two numbers
    int sum = num1 + num2;

    // Print the results to the Serial Monitor (optional)
    Serial.print("Number 1: ");
    Serial.println(num1);
    Serial.print("Number 2: ");
    Serial.println(num2);
    Serial.print("Sum: ");

```

```
Serial.println(sum);
```

```
// Do nothing for a short period to avoid spamming the Serial Monitor (optional)  
delay(5000);  
}
```

Output:

Program to Calculate the Average of Three Numbers:

```
void setup() {  
  // Initialize serial communication for debugging (optional)  
  Serial.begin(9600);  
}
```

```
void loop() {  
  // Define three variables to hold the numbers  
  int num1 = 10;  
  int num2 = 20;  
  int num3 = 30;
```

```
  // Calculate the average of the three numbers  
  float average = (num1 + num2 + num3) / 3.0;
```

```
  // Print the results to the Serial Monitor (optional)  
  Serial.print("Number 1: ");  
  Serial.println(num1);  
  Serial.print("Number 2: ");  
  Serial.println(num2);  
  Serial.print("Number 3: ");  
  Serial.println(num3);  
  Serial.print("Average: ");  
  Serial.println(average);
```

```
  // Do nothing for a short period to avoid spamming the Serial Monitor (optional)  
  delay(5000);  
}
```

Output:

Program to Find the Maximum of Two Numbers:

```
void setup() {  
  // Initialize serial communication for debugging (optional)  
  Serial.begin(57600);  
}  
  
void loop() {  
  // Define two variables to hold the numbers  
  int num1 = 10;  
  int num2 = 20;  
  
  // Find the maximum of the two numbers  
  int maxNum = max(num1, num2);  
  
  // Print the results to the Serial Monitor (optional)  
  Serial.print("Number 1: ");  
  Serial.println(num1);  
  Serial.print("Number 2: ");  
  Serial.println(num2);  
  Serial.print("Maximum: ");  
  Serial.println(maxNum);  
  
  // Do nothing for a short period to avoid spamming the Serial Monitor (optional)  
  delay(5000);  
}
```

Output:

Pre-lab questions

1. What is the purpose of an embedded C program performing basic arithmetic operations?
2. Can you provide an example of a basic arithmetic operation that can be performed using an embedded C program?
3. How do you declare variables in an embedded C program for arithmetic operations?
4. What are the basic arithmetic operators available in embedded C?
5. Can you explain the order of precedence of arithmetic operators in embedded C?

Post Lab Questions

1. How do you perform addition and subtraction operations in an embedded C program?
2. How do you perform multiplication and division operations in an embedded C program?
3. Are there any additional libraries or headers that need to be included for performing arithmetic operations in embedded C?
4. What are the potential issues or limitations when performing arithmetic operations in embedded C?
5. Can you provide an example of an embedded C program that performs a complex arithmetic operation?

Result:

Thus the embedded C programs to perform basic and arithmetic Programs are verified.

EXP.NO: 05

DATE:

Introduction to Arduino platform and programming

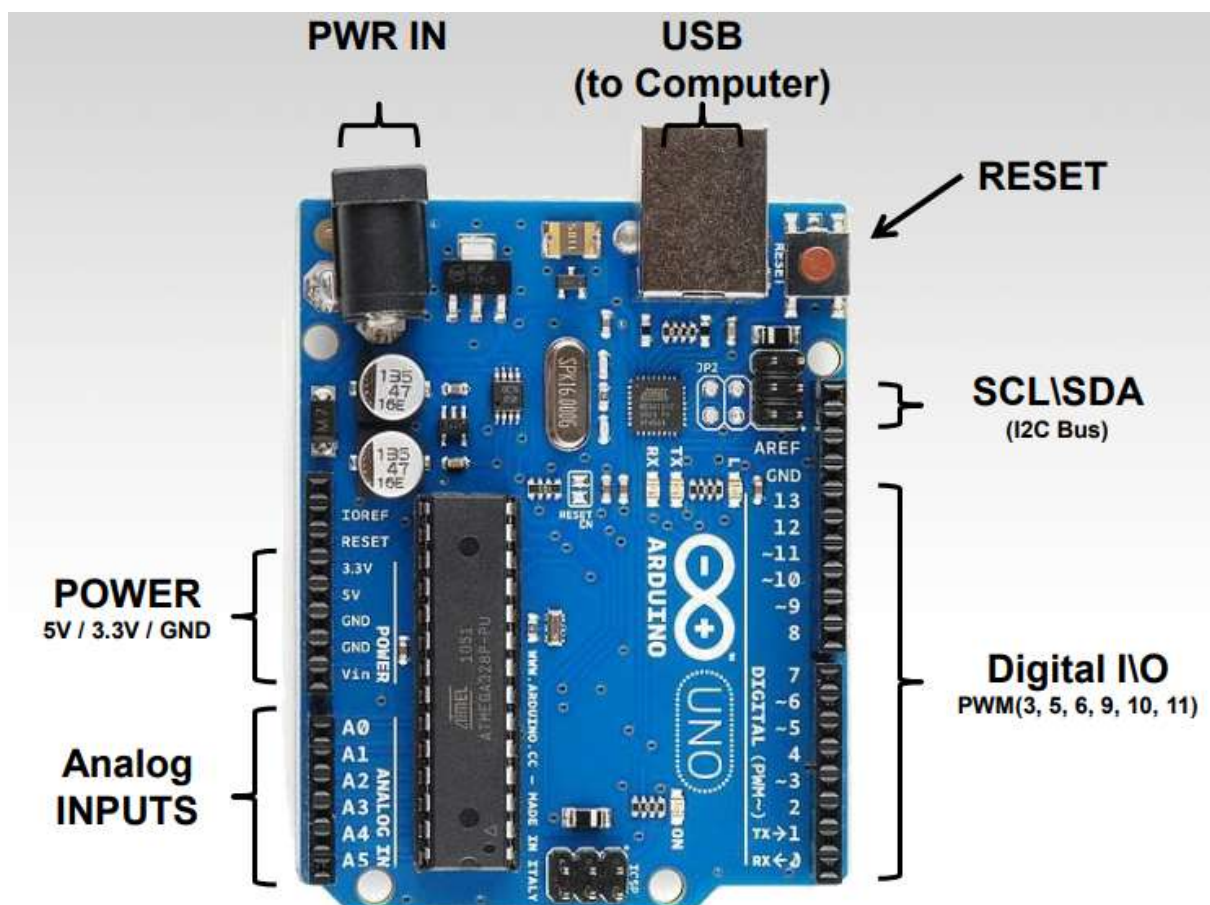
Aim:

To study the Arduino platform and write the Arduino program to blink the LED.

COMPONENTS NEEDED:

1. Arduino UNO development board
2. LED
3. 220Ω resistor
4. Arduino to PC power cable
5. Breadboard
6. Connecting wires

Theory:



TECHNICAL SPECIFICATIONS OF ARDUINO UNO:

Microcontroller - ATmega328

Operating Voltage - 5V

Supply Voltage (recommended) - 7-12V

Maximum supply voltage (not recommended) - 20V

Digital I/O Pins - 14(of which 6 provide PWM o/p)

Analog Input Pins – 6

DC Current per I/O Pin - 40 mA

DC Current for 3.3V Pin - 50 mA

Flash Memory - 32 KB (ATmega328) of which 0.5KB Used by Bootloader

SRAM - 2 KB (ATmega328)

EEPROM - 1 KB (ATmega328)

Clock Speed - 16 MHz

Component Explanations:

- **Analog input pins** – pins (A0-A5) that take-in analog values to be converted to be represented with a number range 0-1023 through an Analog to Digital Converter (ADC).
- **ATmega328 chip** – 8-bit microcontroller that processes the sketch you programmed.
- **Built-in LED** – an on board LED to pin 13.
- **Crystal Oscillator** – clock that has a frequency of 16MHz
- **DC Jack** – where the power source (AC-to-DC adapter or battery) should be connected. It is limited to input values between 6-20V but recommended to be around 7-12V.
- **Digital I/O pins** – input and output pins (0-13) of which 6 of them (3, 5, 6, 9, 10 and 11) also provide PWM (Pulse Width Modulated) output by using the analogWrite() function. Pins (0 (RX) and 1 (TX)) are also used to transmit and receive serial data.
- **ICSP Header** – pins for “In-Circuit Serial Programming” which is another method of programming.

- **ON indicator** – LED that lights up when the board is connected to a power source.
- **Power Pins** – pins that can be used to supply a circuit with values VIN (voltage from DC Jack), 3.3V and 5V.
- **Reset Button** – A button that is pressed whenever you need to restart the sketch programmed in the board.
- **USB port** – allows the user to connect with a USB cable the board to a PC to upload sketches or provide a voltage supply to the board. This is also used for serial communication through the serial monitor from the Arduino software.

Integrated Development Environment (IDE):

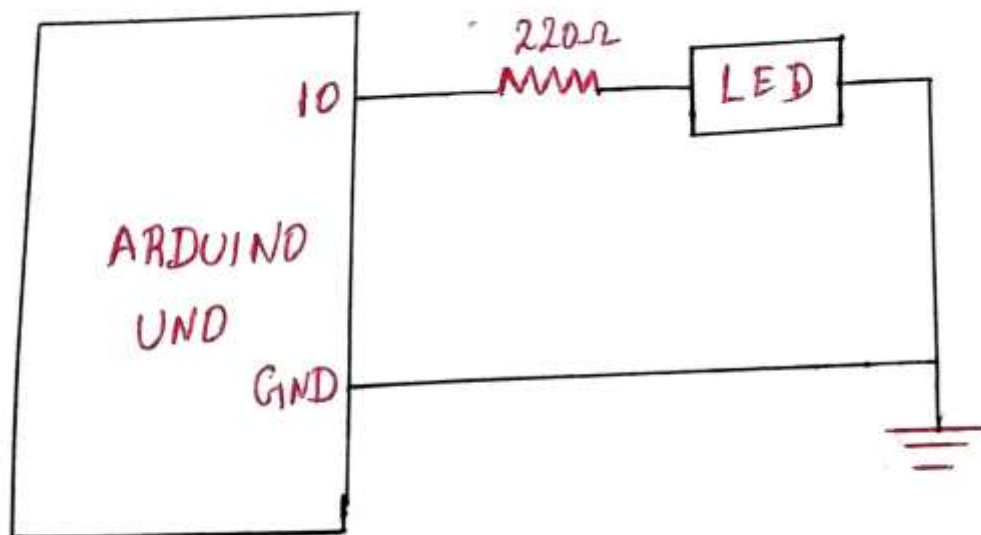
- Once the board is installed, it's time to open the Arduino IDE.
- It is fairly simple when compared to larger desktop C language development systems.



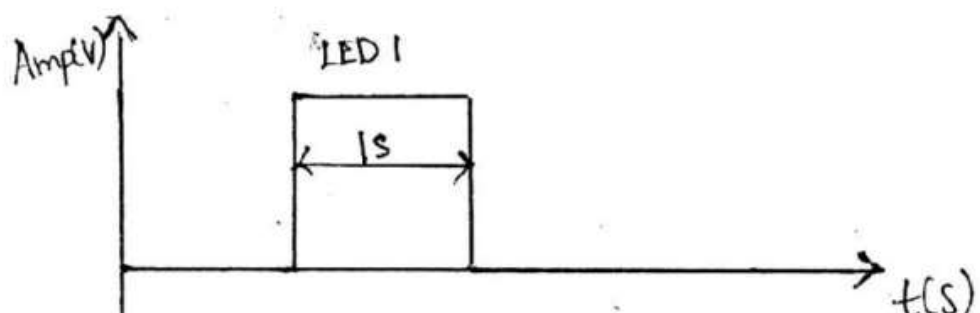
PROGRAMMING:

```
void setup()
{
  // initialize digital pin 10 as an output.
  pinMode(10, OUTPUT);
}
// the loop function runs over and over again forever
void loop()
{
  digitalWrite(10, HIGH); // turn the LED on (HIGH is the voltage level)
  delay(1000);             // wait for a second
  digitalWrite(10, LOW);  // turn the LED off by making the voltage LOW
  delay(1000);             // wait for a second
}
```

CONNECTION DIAGRAM:



MODEL GRAPH:



PROCEDURE:

1. Arduino development board is connected to PC through USB cable
2. Open “Arduino” icon from the computer desktop
3. Select tools menu to choose Arduino board type
4. Select tools menu to choose Arduino board COM port
5. Select file menu to choose new and type program
6. Click Verify icon to once compilation done to upload the program to Arduino controller by clicking Upload icon
7. The circuit connections were given as shown in connection diagram.
8. When the given LED glows with a respective time delay.

Result:

Thus the study of the Arduino platform and the Arduino program to blink LED has been studied and verified successfully.

Ex no:06	Explore different communication methods with IoT devices
Date:	

Aim

To explore different communication methods with IoT devices (Zigbee, GSM, Bluetooth) Arduino programs to communicate with IoT devices using Zigbee, GSM, and Bluetooth

Apparatus Required

1. Personal computer
2. Arduino Uno board
3. Arduino simulator
4. Zigbee, GSM and Bluetooth board

Zigbee Communication:

```
#include <SoftwareSerial.h>
```

```
SoftwareSerial zigbee(2, 3); // RX, TX
```

```
void setup() {  
  Serial.begin(9600);  
  zigbee.begin(9600);  
}
```

```
void loop() {  
  if (zigbee.available()) {  
    char receivedData = zigbee.read();  
    Serial.print("Received data: ");  
    Serial.println(receivedData);  
  }  
}
```

```
  if (Serial.available()) {  
    char sendData = Serial.read();  
    zigbee.print(sendData);  
  }  
}
```

Procedure

This program sets up communication between an Arduino and a Zigbee module using SoftwareSerial. It reads data from the Zigbee module and prints it to the serial monitor, and it also allows data to be sent from

the serial monitor to the Zigbee module.

OUTPUT:

GSM Communication (Sending SMS):

```
#include <SoftwareSerial.h>
```

```
SoftwareSerial gsm(2, 3); // RX, TX
```

```
void setup() {  
  Serial.begin(9600);  
  gsm.begin(9600);  
  delay(2000);  
  gsm.println("AT+CMGF=1"); // Set SMS mode to text  
  delay(1000);  
  gsm.println("AT+CMGS=\"+1234567890\""); // Replace with desired phone number  
  delay(1000);  
  gsm.println("Hello from Arduino!"); // Replace with desired message  
  delay(1000);  
  gsm.println((char)26); // Send Ctrl+Z to end the message  
}  
  
void loop() {  
  // Empty loop  
}
```

Procedure

This program uses SoftwareSerial to establish communication between an Arduino and a GSM module. It sends an SMS message containing a predefined message to a specified phone number.

OUTPUT:

Bluetooth Communication:

```
#include <SoftwareSerial.h>

SoftwareSerial bluetooth(2, 3); // RX, TX

void setup() {
  Serial.begin(9600);
  bluetooth.begin(9600);
}

void loop() {
  if (bluetooth.available()) {
    char receivedData = bluetooth.read();
    Serial.print("Received data: ");
    Serial.println(receivedData);
  }

  if (Serial.available()) {
    char sendData = Serial.read();
    bluetooth.print(sendData);
  }
}
```

Procedure

This program establishes communication between an Arduino and a Bluetooth module using SoftwareSerial. It reads data from the Bluetooth module and prints it to the serial monitor, and it also allows data to be sent from the serial monitor to the Bluetooth module.

Make sure to adjust the pins used for SoftwareSerial communication and modify the specific settings based on your IoT devices' requirements. Additionally, ensure that you have the necessary libraries installed for Zigbee, GSM, or Bluetooth communication.

OUTPUT:

Pre-lab questions:

1. What is the purpose of using Zigbee, GSM, and Bluetooth modules in Arduino projects?
2. How can you establish communication between an Arduino and a Zigbee module?
3. Which library is commonly used for Zigbee communication in Arduino programs? How do you use it?
4. How can you send and receive data between an Arduino and a Zigbee module?
5. Can you explain the process of establishing GSM communication with an Arduino?

Post Lab Questions:

1. What are the AT commands used for GSM communication in Arduino programs?
2. How can you send an SMS message using an Arduino and a GSM module?
3. Which library is commonly used for GSM communication in Arduino programs? How do you use it?
4. How do you establish Bluetooth communication between an Arduino and a Bluetooth module?
5. What are the key functions used for Bluetooth communication in Arduino programs?

Result:

Thus the Arduino programs to communicate with IoT devices (Zigbee, GSM, Bluetooth) are written and verified for its execution